The Globe oposiciones

Bloque 3 - Tema 6

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS

Resumen C#

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE **2**

1. INTRODUCCIÓN 3
   1. *Palabras reservadas del lenguaje* *3*
   2. *Identificadores* *4*
   3. *Variables, tipos de datos y constantes* *4*
2. OPERADORES 9
3. ESTRUCTURA DE UN PROGRAMA EN C# 12
4. ESTRUCTURAS CONDICIONALES Y REPETITIVAS 13
   1. *Sentencias if* *13*
   2. *Sentencias switch* *13*
   3. *Bucle for* *14*
   4. *Bucle foreach* *14*
   5. *Bucle while* *15*
   6. *Bucle do/while* *15*
   7. *Sentencias break y continue* *15*
   8. *return* *16*
5. ARRAYS, CADENAS Y FECHAS 17
   1. *Arrays* *17*
   2. *Cadenas* *18*
   3. *Fechas* *19*
6. CLASES 20
   1. *Constructores de una clase* *24*
   2. *Campos* *26*
   3. *Clases genéricas* *27*
   4. *Eliminación de objetos* *27*
7. HERENCIA 28
   1. *Constructores y herencia* *28*
   2. *Redefinición de métodos* *29*
   3. *Clases abstractas* *31*
   4. *Métodos abstractos* *31*
   5. *Clases y métodos no heredables* *32*
8. INTERFACES 33
   1. *Definición de una interface* *33*
   2. *Herencia en interfaces* *34*
   3. *Interfaces genéricas* *34*
9. EXCEPCIONES 35
   1. *Creación de una excepción* *35*
10. ESPACIOS DE NOMBRES 37
11. ENTRADA / SALIDA 38

1. INTRODUCCIÓN

C# es un lenguaje de programación multiparadigma, compilado, orientado a objetos y con seguridad de tipos desarrollado por Microsoft. La última versión estable es la 14 (2025).

C# tiene sus raíces en la familia de lenguajes C. Está estandarizado por **ECMA-334** e ISO/IEC 23270. El compilador de C# de Microsoft para .NET Framework es una implementación compatible de ambos estándares.

Destacan las siguientes características de este lenguaje:

- C# es un lenguaje **orientado a objetos**, pero también incluye compatibilidad para programación orientada a componentes.
- C# tiene un **sistema de tipo unificado**. Todos los tipos de C#, incluidos los tipos primitivos como `int` y `double`, se heredan de un único tipo **object** raíz.
- El control de versiones asegura que los programas y las bibliotecas de C# pueden evolucionar a lo largo del tiempo de manera compatible. Se ha puesto mucho énfasis en el **versionamiento** del diseño de C#.
- Soporte de expresiones lambda que admiten técnicas de **programación funcional**.
- **Control de excepciones** proporcionando un enfoque estructurado y extensible para la detección y recuperación de errores.
- **Recolección de elementos no utilizados** de forma automática liberando la memoria que ocupan.
- La sintaxis de Language Integrated Query (**LINQ**) crea un patrón común para trabajar con datos de cualquier origen.
- La compatibilidad del lenguaje con las **operaciones asincrónicas** proporciona la sintaxis para crear sistemas distribuidos.

1. Palabras reservadas del lenguaje

El lenguaje C# posee un número amplio de palabras reservadas (77) que define el estándar ANSI-C:

PALABRAS RESERVADAS							
abstract	as	base	bool	break	byte	case	catch
char	checked	class	const	continue	decimal	default	delegate
do	double	else	enum	event	explicit	extern	false
finally	fixed	float	for	foreach	goto	if	implicit
in	int	interface	internal	is	lock	long	namespace
new	null	object	operator	out	override	params	private
protected	public	readonly	ref	return	sbyte	sealed	short
sizeof	stackalloc	static	string	struct	switch	this	throw
true	try	typeof	uint	ulong	unchecked	unsafe	ushort
using	virtual	void	volatile	while			

Las palabras reservadas se escriben siempre minúsculas.

2. Identificadores

C# es un lenguaje sensible al contexto, diferencia entre mayúsculas y minúsculas.

Un **identificador** es cualquier palabra no reservada que comience por una letra o por un subrayado, pudiendo contener en su interior letras, números y subrayados.

Se puede utilizar como identificador una palabra reservada anteponiéndole `@`.

Ejemplo:

Identificadores correctos: `x`, `_y`, `y22`, `VARIABLE`, `tanto_por_ciento`, `@if`, `@for`
Identificadores erróneos: `5d`, `"44`, `por-ciento`, `por ciento`

3. Variables, tipos de datos y constantes

Hay 2 tipos de variables en C#:

- **Tipos de VALOR**: contienen directamente los datos. Cada variable tiene su propia copia de los datos y no es posible que las operaciones en una variable afecten a la otra (salvo variables de parámetro `ref` y `out`).

  Los tipos de valor de C# se dividen en:

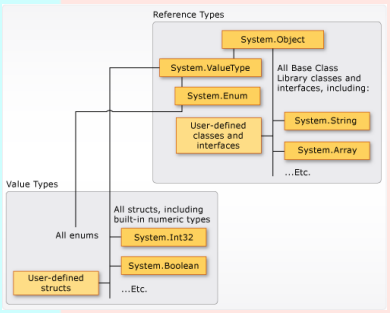
  - Tipos simples.
  - Tipos de enumeración.
  - Tipos de estructura (`struct`).
  - Tipos que aceptan valores NULL.

Tipos de valor	Descripción	Tipos
Simples	Enteros con signo	`sbyte, short, int, long`
	Enteros sin signo	`byte, ushort, uint, ulong`
	Caracteres Unicode (UTF-16)	`char`
	Punto flotante binario	`float, double`
	Punto flotante decimal alta precisión	`decimal`
	Booleano	`bool`
De enumeración	Conjunto de constantes	`enum` Ej: `enum E {…}`
De estructura	Encapsula datos y funcionalidad	`struct` Ej: `struct S {…}`
Aceptan valores NULL (anulables)	Extensión de todos los demás tipos	
De tupla	Agrupa varios elementos	Ej: `(double, int) t1 = (4.5, 3)`

- **Tipos de REFERENCIA**: almacenan referencias a los datos (objetos). Es posible que dos variables hagan referencia al mismo objeto y que, por tanto, las operaciones en una variable afecten al objeto al que hace referencia la otra variable.

Los tipos de referencia se dividen en:

- Tipos de clase.
- Tipos de interfaz.
- Tipos de matriz.
- Tipos delegado.



Tipos de referencia	Descripción	Tipos
De clase	Clase base	`Object`
	Cadenas	`String`
	Clases definidas por el usuario	`class C {…}`
De interfaz	Conjunto de métodos (igual que en Java)	`interface I {…}`
De matriz	Unidimensionales o multidimensionales	`tipo []` `tipo [,…,]`
Delegado	Declaración de métodos. Es como un puntero a función	`delegate tipo D(…)`

La sintaxis de declaración una variable tiene el siguiente formato:

```
<tipo_dato> <nombre_variable> [, <nombre_variable];
```

TIPO	DESCRIPCIÓN	CLASE	MÍNIMO	MÁXIMO
sbyte	Entero de 8 bits con signo	SByte	-128	127
byte	Entero de 8 bits sin signo	Byte	0	255
short	Entero de 16 bits con signo	Int16	-32768	32767
ushort	Entero de 16 bits sin signo	UInt16	0	65535
int	Entero de 32 bits con signo	Int32	-2^{31}	$2^{31}-1$
uint	Entero de 32 bits sin signo	UInt32	0	$2^{32}-1$
long	Entero de 64 bits con signo	Int64	-2^{63}	$2^{63}-1$
ulong	Entero de 64 bits sin signo	UInt64	0	$2^{64}-1$
char	16 bits. Un carácter se puede especificar con un literal, con la representación Unicode precedido de \u o con la representación hexadecimal precedido de \x	Char		
float	Flotante de 32 bits	Single		
double	Flotante de 64 bits	Double		
decimal	Numérico de 128 bits Para cálculos financieros y monetarios	Decimal		
bool	8 bits. Valores lógicos `false` o `true`. No conversión entre `bool` y tipos enteros	Boolean		

Ejemplos: `int a=4;` `char asterisco='*';` `float suma=0.11;`

Las variables se pueden categorizar en:

- **Variables estáticas**: en la declaración se incluye el modificador **`static`**.
- **Variables de instancia**: cualquier variable sin el modificador `static`.

Las variables que se declaran en el ámbito de método pueden tener un "tipo" **`var`** implícito. Una variable local con tipo implícito es fuertemente tipada exactamente igual que si hubiera declarado el tipo, solo que en este caso es el compilador infiere el tipo.

Ejemplos:

```
var i = 10; // Implicitly typed - System.Int32
int i = 10; // Explicitly typed - System.Int32
var greeting = "Hello"; // Implicitly typed - System.String
```

Sobre los tipos que aceptan valores NULL, ayuda a que en tiempo de ejecución no se produzcan excepciones por referencias nulas (`System.NullReferenceException`). Los tipos anulables se declaran de la siguiente forma:

```
tipo? variable;
```

Lo que está haciendo el compilador es traducir a:

```
System.Nullabe<tipo> variable;
```

En C# se pueden definir constantes anteponiendo en la declaración de la variable la palabra reservada **const**.

Ejemplos:

```
const double Pi = 3.14159;
const int SpeedOfLight = 300000;
```

Ciertos caracteres no imprimibles, como las comillas dobles ("), el apóstrofo o comilla simple ('), etc. Se pueden expresar en términos de una **secuencia de escape**:

CÓDIGO	SIGNIFICADO
\b	retroceso (backspace)
\f	avance de página
\n	nueva línea
\r	retorno de carro
\t	tabulador horizontal
\"	comilla doble
\'	comilla simple
\0	nulo
****	barra invertida
\v	tabulador vertical
\a	alarma
\ ooo	carácter ASCI en octal
\x hhhh	carácter Unicode en hexadecimal

Conversión AUTOMÁTICA de tipos

La conversión implícita se considera segura, no ocasionando pérdida de información. Se incluye la conversión de tipos enteros más pequeños a más grandes y conversiones de clases derivadas a clases base.

Conversión EXPLÍCITA de tipos

Opción de forzar la conversión de un tipo de datos en otro tipo de datos. Esta conversión de un tipo de datos en otro se llama "casting", y su sintaxis es:

```
(tipo) expresión
```

Ejemplo:

```
int a=3,b=2;
float c;
c=a/b;
```

La operación asigna a `c` el valor 1.0 en vez de el valor 1.5, porque a y b son variables de tipo entero. Se realiza una división entre enteros, y el resultado de 3/2 es 1. A continuación ese valor 1 se convierte a un valor en coma flotante para realizar la asignación (valor 1.0), y se asigna a `c`. Si lo que se desea es que la división se realice en punto flotante, debe escribirse la operación de la siguiente forma:

```
c=(float)a/b;
```

Esta conversión obliga a convertir la variable a en `float`, y, aplicando las reglas de conversión automática de tipos, se realiza la división en coma flotante, dando lugar a que el resultado de la operación sea 1.5, y dicho valor sea el asignado a la variable `c`.



2. OPERADORES

TIPO	OPERADOR	SIGNIFICADO
Aritméticos	**+**	Adición
	-	Sustracción
	*****	Multiplicación
	/	División
	%	Resto de división entera
	++	Incremento (operador unario)
	--	Decremento (operador unario)
Comparación	**>**	Mayor que
	>=	Mayor o igual que
	<	Menor que
	<=	Menor o igual que
Igualdad	**==**	Igual
	!=	Distinto
Lógicos	**&**	AND lógico
	&&	AND lógico condicional. En `x&&y` si `x` es `false`, y no se evalúa
	|	OR lógico
	||	OR lógico condicional. En `x||y` si `x` es `true`, y no se evalúa
	!	NOT lógico
	^	XOR lógico
Asignación	**=**	Asignación simple
	***=**	Asignación de la multiplicación
	/=	Asignación de la división
	%=	Asignación del resto
	+=	Asignación de la suma
	-=	Asignación de la resta

Tratamiento de bits	**|**	Operación OR (operandos de tipo entero)
	^	Operación XOR (operandos de tipo entero)
	&	Operador AND (operandos de tipo entero)
	>>	Desplazamiento a la derecha
	<<	Desplazamiento a la izquierda
	~	Complemento (operador unario). Invierte cada bit
	.	Acceso a los campos si no hay puntero
	[]	Acceso a los elementos de una matriz
	()	Modificación orden evaluación expresiones
	new	Crea una instancia de un tipo
	sizeof	Número de bytes de una variable de un tipo `sizeof(float), sizeof(int)`
	is	Comprobación de si el resultado de una expresión es compatible con un tipo determinado `variable is int`
	delegate	Crea un método anónimo que se puede convertir en un tipo delegado
	?	Operador condicional ternario Condición ? true : false (operador ternario)
	??	Operador de uso combinado NULL Devuelve el valor del operando izqdo. si no es null; en otro caso, evalúa el operando derecho y devuelve el resultado. `a ?? b`
	??=	Operador de asignación de uso combinado NULL Asigna el valor del operando derecho al operando izqdo. solo el operando izqdo. se evalúa como null; no se evalúa el operando derecho y si el operando izqdo. se evalúa como no null. `d ??= e`
	=>	Operador lambda

PRECEDENCIA OPERADORES
() [] -> .
! ~ ++ --
- * &
sizeof
* / %
+ -
<< >>
< <= >= >
== !=
&
^
|
&&
||
?:
= += -= *= /= %=
&= ^= |= <<= >>=

3. ESTRUCTURA DE UN PROGRAMA EN C#

Un programa en C# consta de uno o más archivos. Cada archivo contiene cero o más espacios de nombres. Un espacio de nombres contiene tipos como clases, estructuras, interfaces, enumeraciones u otros espacios de nombres. El siguiente ejemplo es el esqueleto de un programa de C# que contiene todos estos elementos:

```
// A skeleton of a C# program
using System;

// Your program starts here:
Console.WriteLine("Hello world!");

namespace YourNamespace {
    class YourClass { }

    struct YourStruct { }

    interface IYourInterface { }

    delegate int YourDelegate();

    enum YourEnum { }

    namespace YourNestedNamespace {
        struct YourStruct { }
    }
}
```

En el ejemplo anterior se usan **instrucciones de nivel superior** para el punto de entrada del programa. Esta característica se agregó en C# 9. Antes de C# 9, el punto de entrada era un método estático denominado **Main**, como se muestra en el ejemplo siguiente:

```
class Program {
        static void Main(string[] args) {
            //Your program starts here...
            Console.WriteLine("Hello world!");
        }
}
```

4. ESTRUCTURAS CONDICIONALES Y REPETITIVAS

1. Sentencias if

Tres formas:

FORMA 1
```
if(condición)
       sentencia;
```

FORMA 2
```
if(condición)
       sentencia1;
else
       sentencia2;
```

FORMA 3
```
if(condición)
       sentencia1;
else if(condición)
       sentencia2;
else
       sentencia3;
```

Ejemplo:

```
if (char.IsUpper(ch)) {
      Console.WriteLine($"An uppercase letter: {ch}");
}
else if (char.IsLower(ch)) {
      Console.WriteLine($"A lowercase letter: {ch}");
}
else if (char.IsDigit(ch)) {
      Console.WriteLine($"A digit: {ch}");
}
else {
      Console.WriteLine($"Not alphanumeric character: {ch}");
}
```

2. Sentencias switch

Selecciona una lista de instrucciones para ejecutarla en función de la coincidencia de un patrón con una expresión de coincidencia. La forma general de la sentencia **`switch`** es:

```
switch(variable) {
      case const1:
            sentencia;
            break;
      case const2:
            sentencia;
            break;
      ...
      default:
            sentencia;
}
```

El caso `default` puede aparecer en cualquier lugar de una instrucción `switch`. Independientemente de su posición, el caso `default` siempre se evalúa por último y solo si no coinciden todos los demás patrones de caso.

Ejemplo:

```
DisplayMeasurement(-4);  // Output: Measured value is -4; too low.
DisplayMeasurement(5);  // Output: Measured value is 5.
DisplayMeasurement(30);  // Output: Measured value is 30; too high.
DisplayMeasurement(double.NaN);  // Output: Failed measurement.

void DisplayMeasurement(double measurement) {
    switch (measurement) {
        case < 0.0:
            Console.WriteLine("Measured value is {measurement}; too low.");
            break;

        case > 15.0:
            Console.WriteLine("Measured value is {measurement}; too high.");
            break;

        case double.NaN:
            Console.WriteLine("Failed measurement.");
            break;

        default:
            Console.WriteLine("Measured value is {measurement}.");
            break;
    }
}
```

3. Bucle for

La sintaxis del bucle for es:

```
for(inicialización; condición; incremento) sentencia;
```

4. Bucle foreach

Permite iterar sobre una colección de elementos.

```
foreach(variable in coleccion) {
      sentencia;
}
```

Ejemplo:

```
var fibNumbers = new List<int> { 0, 1, 1, 2, 3, 5, 8, 13 };
foreach (int element in fibNumbers) {
    Console.Write($"{element} ");
}

// Output:
// 0 1 1 2 3 5 8 13

int[,] numbers2D = new int[3, 2] { { 9, 99 }, { 3, 33 }, { 5, 55 } };
// Or use the short form:
// int[,] numbers2D = { { 9, 99 }, { 3, 33 }, { 5, 55 } };
```

```
foreach (int i in numbers2D) {
    System.Console.Write("{0} ", i);
}
// Output: 9 99 3 33 5 55
```

5. Bucle while

La sintaxis del bucle while es:

```
while (condición) sentencia;
```

Donde la sentencia puede no existir (sentencia vacía), pero siempre debe existir la condición.

Ejemplo:

```
int n = 0;
while (n < 5) {
    Console.Write(n);
    n++;
}
// Output:
// 01234
```

6. Bucle do/while

La sintaxis del bucle do/while es:

```
do
    sentencia;
while (condición);
```

7. Sentencias break y continue

Sentencias:

- **`break`**: permite salir de cualquier bloque. Se utiliza para interrumpir un bucle `while`, `do`, `for` o una sentencia `switch`.
- **`continue`**: salta al final del cuerpo de un bucle `while`, `do` o `for`, evaluándose la expresión `boolean` de control.

Ejemplo: instrucción `break`

```
class BreakTest {
    static void Main() {
        for (int i = 1; i <= 100; i++) {
            if (i == 5) {break;}
            Console.WriteLine(i);
        }
```

```
        // Keep the console open in debug mode.
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
/*
 Output:
    1
    2
    3
    4
*/
```

Ejemplo: instrucción `continue`

```
class ContinueTest {
    static void Main() {
        for (int i = 1; i <= 10; i++) {
            if (i < 9) {continue;}
            Console.WriteLine(i);
        }

        // Keep the console open in debug mode.
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
/*
Output:
9
10
*/
```

8. return

Termina la ejecución de un método y puede devolver un valor de retorno. Es obligatorio si el método no es `void`.

```
return [expresión]
```

Ejemplo:

```
class ReturnTest {
    static double CalculateArea(int r) {
        double area = r * r * Math.PI;
        return area;
    }
    static void Main() {
        int radius = 5;
        double result = CalculateArea(radius);
        Console.WriteLine("The area is {0:0.00}", result);
        // Keep the console open in debug mode.
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
// Output: The area is 78.54
```

5. ARRAYS, CADENAS Y FECHAS

1. Arrays

Estructura de datos que almacena múltiples valores del mismo tipo. El tamaño del array se determina en la creación del array no pudiendo modificarse después. En C#, las matrices son actualmente objetos, y no simplemente regiones direccionables de memoria contigua como en C y C++. **Array** es el tipo base abstracto de todos los tipos de arrays. Sintaxis de la declaración de un array:

```
<tipo>[] <nombre_array>;
```

El primer elemento de un array es el que posee el índice 0 y el último, tamaño-1. Si se quiere que el array almacene elementos de cualquier tipo, se especifica `object` como su tipo.

Ejemplo: `String[] titulos; Double[] precios; object[] elem;`

Para la creación de un array utilizamos el operador **new**. Los elementos son inicializados automáticamente a sus valores por defecto.

```
new <tipo>[<tamaño>];
```

Ejemplos:

```
titulos = new String[10];
Double[] notas = new double[20];
int[,] array = new int[4, 2];
int[,,] array1 = new int[4, 2, 3];
```

Es posible la creación e inicialización en un único paso. El tamaño del array viene determinado por el número de elementos especificados entre las llaves.

```
<tipo>[] <nombre_array> = new <tipo>[]{<lista elementos>};
```

Ejemplos:

```
int[] array1 = new int[] { 1, 3, 5, 7, 9 };
string[] weekDays=new string[] {"Sun","Mon","Tue","Wed","Thu","Fri","Sat"};

// 2 dimensiones
int[,] array2D = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };

// 2 dimensiones especificadas
int[,] array2Da = new int[4, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };

string[,] array2Db = new string[3, 2]{{"one","two"}, {"three","four"}, {"five",
"six"}};

// 3 dimensiones
int[,,] array3D = new int[,,] { { { 1, 2, 3 }, { 4, 5, 6 } },
                                { { 7, 8, 9 }, { 10, 11, 12 } } };
```

Se puede evitar la expresión `new` y el tipo de array al inicializarlo:

```
int[] array2 = { 1, 3, 5, 7, 9 };
string[] weekDays2 = { "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat" };
```

Propiedades:

- **`Length`**: número de elementos de todas las dimensiones.
- **`Rank`**: obtiene el rango, es decir, el número de dimensiones. Array unidimensional devuelve 1, array bidimensional 2, etc.

Métodos:

- **`Clear(Array, inicio, fin)`**: asigna el valor predeterminado a un rango.
- **`Copy(ArrayOrigen, ArrayDestino, numero)`**: copia `numero` de elementos del origen al destino.
- **`GetValue(int)`**: o `GetValue(int,int)` o `GetValue(int,int,int)` devuelve el elemento de la posición especificada.
- **`Sort(Array)`**: ordena un array.
- **`ToString()`**: devuelve una cadena que representa el array.

2. Cadenas

Una cadena es un objeto de tipo **`String`** cuyo valor es texto. Internamente, el texto se almacena como una colección secuencial de solo lectura de objetos `Char`.

La propiedad `Length` de una cadena representa el número de objetos `Char` que contiene.

En C#, la palabra clave `string` es un alias de `String`. Por lo tanto, `String` y `string` son equivalentes, aunque se recomienda usar el alias proporcionado `string`.

Ejemplos:

```
// Declare without initializing.
string message1;

// Initialize to null.
string message2 = null;

// Initialize as an empty string.
// Use the Empty constant instead of the literal "".
string message3 = System.String.Empty;

// Initialize with a regular string literal.
string oldPath = "c:\\Program Files\\Microsoft Visual Studio 8.0";

// Initialize with a verbatim string literal.
string newPath = @"c:\Program Files\Microsoft Visual Studio;
```

Los objetos de cadena son **inmutables**: no se pueden cambiar después de haberse creado. Todos los métodos `String` y operadores de C# que parecen modificar una cadena en realidad devuelven los resultados en un nuevo objeto de cadena.

Propiedades:

- **`Length`**: número de elementos de todas las dimensiones.

Métodos:

- **`Compare(String, String)`**: compara 2 cadenas.
- **`Equals(String)`**: determina si un objeto `String` tienen el mismo valor que la instancia.
- **`Format`**: permite insertar el valor de un objeto, variable o expresión en otra cadena.
- **`IndexOf(char)`**: devuelve el índice de la primera aparición de `char` en la cadena.
- **`LastIndexOf(char)`**: devuelve el índice de la última aparición de `char` en la cadena.
- **`Replace(String old, String new)`**: reemplaza en la cadena actual todas las apariciones de `old` por `new`.
- **`Split(Char)`**: divide una cadena en subcadenas en función del carácter delimitador.
- **`Trim(Char)`**: elimina todas las instancias iniciales y finales de un carácter de la cadena actual.
- **`ToLower()`**: devuelve una copia de la cadena convertida en minúsculas.
- **`ToUpper()`**: devuelve una copia de la cadena convertida en mayúsculas.

3. Fechas

`DateTime` representa un instante de tiempo, normalmente expresado en forma de fecha y hora del día.

Ejemplos:

```
DateTime date1 = DateTime.Now;
DateTime date2 = DateTime.UtcNow;
DateTime date3 = DateTime.Today;
```

Métodos a destacar:

- **`Parse(String)`**: convierte una cadena en su valor de fecha y hora equivalente.
- **`ToString()`**: devuelve la representación de cadena de un valor de fecha y hora.

6. CLASES

Las clases se declaran mediante la palabra clave **class**:

```
[public | protected | internal | private] [abstract | sealed | static]
class NombreClase [: clasebase] [, interfaz1, interfaz2, ...] {
      // Definición de la clase
}
```

Las clases que se declaran directamente dentro de un espacio de nombres, que no están anidadas dentro de otras clases, pueden ser de tipo `public` o `internal`. Por defecto, las clases son `internal`.

Una clase estática (**static**) es básicamente lo mismo que una clase no estática, con la diferencia de que no se pueden crear instancias de una clase estática.

Ejemplo:

```
public static class TemperatureConverter {…}
```

Los miembros de clase, incluidas las clases anidadas, pueden ser `public`, `protected internal`, `protected`, `internal`, `private` o `private protected`. De forma predeterminada, los miembros son `private`.

El nivel de accesibilidad controla si se pueden usar desde otro código del ensamblado u otros ensamblados. Los modificadores de acceso siguientes para especificar la accesibilidad de un tipo o miembro cuando lo declare:

- **public**: acceso al tipo o miembro cualquier otro código del mismo ensamblado o de otro ensamblado que haga referencia a éste.
- **private**: solamente el código de la misma clase puede acceder al tipo o miembro.
- **protected**: solamente el código de la misma clase, o bien de una clase derivada de esa clase, puede acceder al tipo o miembro.
- **internal**: puede obtener acceso al tipo o miembro cualquier código del mismo ensamblado, pero no de un ensamblado distinto.
- **protected internal**: cualquier código del ensamblado en el que se ha declarado, o desde una clase derivada de otro ensamblado, puede acceder al tipo o miembro.
- **private protected**: el código de la misma clase, o de un tipo derivado de esa clase, puede acceder al tipo o miembro solo dentro de su ensamblado de declaración.

Una clase puede contener (se citan los más relevantes):

- **Campos**: variable declarada en el ámbito de una clase. Los campos pueden ser de instancia o estáticos.

  ```
  private DateTime _date;
  ```

- **Constantes**: campos cuyo valor se establece en tiempo de compilación y no se puede alterar. Se utiliza el calificador **const**.

```
public const int Months = 12;
```

- **Propiedades**: métodos de una clase a los que se tiene acceso como si fueran campos de esa clase. Utiliza descriptores de acceso: **get** (lectura) y **set** (escritura). La palabra clave **value** se puede utilizar y hace referencia al valor que se va a asignar al campo correspondiente.

```
using System;

class TimePeriod {
   private double _seconds;

   public double Hours {
       get { return _seconds / 3600; }
       set {
          if (value < 0 || value > 24)
             throw new ArgumentOutOfRangeException(
                   $"{nameof(value)} must be between 0 and 24.");

          _seconds = value * 3600;
       }
   }
}

class Program {
   static void Main() {
       TimePeriod t = new TimePeriod();
       // The property assignment causes the 'set' accessor to be called.
       t.Hours = 24;

       // Retrieving the property causes the 'get' accessor to be called.
       Console.WriteLine($"Time in hours: {t.Hours}");
   }
}
// The example displays the following output:
//    Time in hours: 24
```

A partir de C# 7.0 los descriptores de acceso se pueden implementar con =>:

```
using System;

public class SaleItem {
   string _name;
   decimal _cost;

   public SaleItem(string name, decimal cost) {
      _name = name;
      _cost = cost;
   }

   public string Name {
      get => _name;
      set => _name = value;
   }
```

```
   public decimal Price {
      get => _cost;
      set => _cost = value;
   }
}

class Program {
   static void Main(string[] args) {
      var item = new SaleItem("Shoes", 19.95m);
      Console.WriteLine($"{item.Name}: sells for {item.Price:C2}");
   }
}
// The example displays output like the following:
//       Shoes: sells for $19.95
```

Por último, las propiedades pueden ser implementadas automáticamente si no se proporciona código a los descriptores de acceso (`get` y `set`), es decir, cuando no es necesario incluir ninguna lógica adicional, solo recuperar el valor del campo o guardar el valor en el campo:

```
using System;

public class SaleItem {
   public string Name
   { get; set; }

   public decimal Price
   { get; set; }
}

class Program {
   static void Main(string[] args) {
      var item = new SaleItem{ Name = "Shoes", Price = 19.95m };
      Console.WriteLine($"{item.Name}: sells for {item.Price:C2}");
   }
}
// The example displays output like the following:
//       Shoes: sells for $19.95
```

- **Métodos**: son las acciones que puede realizar una clase.

```
abstract class Motorcycle {
    // Anyone can call this.
    public void StartEngine() {/* Method statements here */ }

    // Only derived classes can call this.
    protected void AddGas(int gallons) { /* Method statements here */ }

    // Derived classes can override the base class implementation.
    public virtual int Drive(int miles, int speed) {
          /* Method statements here */ return 1; }

    // Derived classes must implement this.
    public abstract double GetTopSpeed();
}
```

```
class TestMotorcycle : Motorcycle {
    public override double GetTopSpeed() {return 108.4;}
    static void Main() {
        TestMotorcycle moto = new TestMotorcycle();

        moto.StartEngine();
        moto.AddGas(15);
        moto.Drive(5, 20);
        double speed = moto.GetTopSpeed();
        Console.WriteLine("My top speed is {0}", speed);
    }
}
```

La definición del método especifica los nombres y tipos de todos los parámetros necesarios.

En relación al paso de parámetros, para tipos de valor se pasa por defecto una copia y para tipos de referencia se pasa una referencia al objeto. Se puede utilizar la palabra clave **ref** para pasar una instancia de un tipo de valor por referencia.

Ejemplo:

```
public class SampleRefType {public int value;}

public static void TestRefType() {
    SampleRefType rt = new SampleRefType();
    rt.value = 44;
    ModifyObject(rt);
    Console.WriteLine(rt.value);
}

static void ModifyObject(SampleRefType obj) {obj.value = 33;}

class PassingValByRef {
    static void SquareIt(ref int x)
    // The parameter x is passed by reference.
    // Changes to x will affect the original value of x.
    {
        x *= x;
        System.Console.WriteLine("The value inside the method: {0}", x);
    }
    static void Main() {
        int n = 5;
        System.Console.WriteLine("The value before calling the method: {0}", n);

        SquareIt(ref n);  // Passing the variable by reference.
        System.Console.WriteLine("The value after calling the method: {0}", n);
        // Keep the console window open in debug mode.
        System.Console.WriteLine("Press any key to exit.");
        System.Console.ReadKey();
    }
}
/* Output:
    The value before calling the method: 5
    The value inside the method: 25
    The value after calling the method: 25
*/
```

El valor devuelto por un método dependerá de si en la declaración del mismo se especifica **`void`** (no devuelve nada) o el tipo concreto.

Ejemplo:

```
class SimpleMath {
    public int AddTwoNumbers(int number1, int number2) {
        return number1 + number2;
    }

    public int SquareANumber(int number) {return number * number;}
}
```

Los métodos estáticos permiten su invocación directamente sin necesidad de instanciar un objeto (`ClassName.MemberName`).

- **Operadores**: son los operadores sobrecargados, que puede personalizarse su implementación. Se utiliza la palabra clave **`operator`** para su declaración. Además, se deben cumplir las siguientes reglas: incluir en la declaración los modificadores `public` y `static` y al menos uno de los parámetros debe ser del tipo de la clase que se está definiendo.

```
public static Fraction operator -(Fraction a) => new Fraction(-a.num, a.den);
public static Fraction operator *(Fraction a, Fraction b)
        => new Fraction(a.num * b.num, a.den * b.den);
```

- **Constructores**: métodos invocados cuando el objeto es creado.

1. Constructores de una clase

Cuando se crea una clase, se llama a su constructor. Los constructores tienen el mismo nombre que la clase y suelen inicializar los miembros de datos del nuevo objeto.

```
public class Taxi {
    public bool IsInitialized;

    public Taxi() {IsInitialized = true;}
}

class TestTaxi {
    static void Main() {
        Taxi t = new Taxi();
        Console.WriteLine(t.IsInitialized);
    }
}
```

Un constructor que no toma ningún parámetro se denomina constructor sin parámetros. Los constructores sin parámetros se invocan cada vez que se crea una instancia de un objeto mediante el operador `new` y no se especifica ningún argumento en `new`.

Una clase puede tener varios constructores (con parámetros diferentes). Un constructor es un método cuyo nombre es igual que el nombre de su tipo. La declaración del método incluye solo el nombre del método y su lista de parámetros; no incluye un tipo de valor devuelto.

```
public class Person {
   private string last;
   private string first;

   public Person(string lastName, string firstName) {
      last = lastName;
      first = firstName;
   }
   // Remaining implementation of Person class.
}

public class Location {
   private string locationName;

   public Location(string name) => Name = name;

   public string Name {
      get => locationName;
      set => locationName = value;
   }
}
```

Una clase también puede tener un constructor estático, que inicializa los miembros estáticos del tipo. Los constructores estáticos no tienen parámetros. Si no se proporciona un constructor estático para inicializar los campos estáticos, el compilador de C# inicializa los campos estáticos a su valor predeterminado:

```
public class Adult : Person {
   private static int minimumAge;

   public Adult(string lastName, string firstName) : base(lastName, firstName)
   { }

   static Adult() {minimumAge = 18;}
   // Remaining implementation of Adult class.
}
```

Las clases estáticas no pueden declarar constructores de instancias.

Un constructor privado es un constructor de instancia especial. Se usa generalmente en clases que contienen solo miembros estáticos. Si una clase tiene uno o más constructores privados y ningún constructor público, el resto de clases (excepto las anidadas) no podrán crear instancias de esta clase:

```
public class Counter {
    private Counter() { }
    public static int currentCount;
    public static int IncrementCount() {return ++currentCount;}
}
```

```
class TestCounter {
    static void Main() {
        // If you uncomment the following statement, it will generate
        // an error because the constructor is inaccessible:
        // Counter aCounter = new Counter();   // Error

        Counter.currentCount = 100;
        Counter.IncrementCount();
        Console.WriteLine("New count: {0}", Counter.currentCount);

        // Keep the console window open in debug mode.
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
// Output: New count: 101
```

2. Campos

Los campos de instancia son específicos de un objeto.

```
Public class CalendarEntry {
    // private field (Located near wrapping "Date" property).
    Private DateTime _date;

    // Public property exposes _date field safely.
    Public DateTime Date {
        get {return _date;}
        set {
            // Set some reasonable boundaries for likely birth dates.
            If (value.Year > 1900 && value.Year <= DateTime.Today.Year) {
                _date = value;
            }
            else {throw new ArgumentOutOfRangeException();}
        }
    }

    // public field (Generally not recommended).
    Public string Day;

    // Public method also exposes _date field safely.
    // Example call: birthday.SetDate("1975, 6, 30");
    public void SetDate(string dateString) {
        DateTime dt = Convert.ToDateTime(dateString);

        // Set some reasonable boundaries for likely birth dates.
        If (dt.Year > 1900 && dt.Year <= DateTime.Today.Year) {_date = dt;}
        else {throw new ArgumentOutOfRangeException();}
    }

    public TimeSpan GetTimeSpan(string dateString) {
        DateTime dt = Convert.ToDateTime(dateString);

        if (dt.Ticks < _date.Ticks) {return _date – dt;}
        else {throw new ArgumentOutOfRangeException();}
    }
}
```

Por el contrario, un campo estático (**static**) pertenece al propio tipo y se comparte entre todas las instancias de ese tipo. Solo puede acceder al campo estático mediante el nombre del tipo. Si obtiene acceso al campo estático mediante un nombre de instancia, obtendrá el error en tiempo de compilación.

```
Public static int NumberOfWheels = 4;
```

3. Clases genéricas

Las clases genéricas encapsulan operaciones que no son específicas de un tipo de datos concreto. El uso más común de las clases genéricas son las colecciones: listas, tablas hash, pilas, colas y árboles.

```
Public class GenericClass<T> {…}
```

Ejemplo: uso de una clase genérica

```
// Declare a list of type int.
GenericList<int> list1 = new GenericList<int>();
```

4. Eliminación de objetos

La instrucción **using** obtiene uno o más recursos, ejecuta las instrucciones especificadas y desecha el objeto automáticamente. Esta instrucción solo resulta útil para los objetos que se usan dentro del ámbito del método en el que se construyen.

```
using (expression)
{
    // ...
}
```

donde expresión genera una instancia que será eliminada al final del bloque de código encerrado entre las llaves. Dicha eliminación supone llamar al método **Dispose** en el objeto creado en expression.

Ejemplos:

```
using (SqlConnection conn = new SqlConnection(connString)) {conn.Open();}

var numbers = new List<int>();
using (StreamReader reader = File.OpenText("numbers.txt")) {
    string line;
    while ((line = reader.ReadLine()) is not null) {
        if (int.TryParse(line, out int number)) {numbers.Add(number);}
    }
}
```

7. HERENCIA

C# admite herencia simple de clases y múltiples interfaces.

La forma completa para la declaración de clases es:

```
modif class nombre [: claseBase, interfaz, …]
```

Ejemplos:

```
class DerivedClass : BaseClass { }

public class BaseClass {
    int num;

    public BaseClass() {Console.WriteLine("in BaseClass()");}

    public BaseClass(int i) {
        num = I;
        Console.WriteLine("in BaseClass(int i)");
    }

    public int GetNum() {return num;}
}

public class DerivedClass : BaseClass
{
    // This constructor will call BaseClass.BaseClass()
    public DerivedClass() : base(){}

    // This constructor will call BaseClass.BaseClass(int i)
    public DerivedClass(int i) : base(i){}

    static void Main() {
        DerivedClass md = new DerivedClass();
        DerivedClass md1 = new DerivedClass(1);
    }
}
/*
Output:
in BaseClass()
in BaseClass(int i)
*/
```

1. Constructores y herencia

Para asegurar la correcta construcción de un objeto, todos los constructores de las superclases del mismo deben invocarse. Para llamar a un constructor de una superclase, utilizamos la palabra reservada **base**.

```
base(parámetros);
```

Ejemplo:

```
public class Manager : Employee {
    public Manager(int annualSalary) : base(annualSalary) {
        //Add further instructions here.
    }
}
```

En una clase derivada, si un constructor de clase base no se llama explícitamente con la palabra clave `base`, se llama implícitamente al constructor sin parámetros, si existe.

Ejemplo:

```
public Manager(int initialData) {
    //Add further instructions here.
}

public Manager(int initialData) : base() {
    //Add further instructions here.
}
```

2. Redefinición de métodos

Una subclase puede anular (proporcionar una versión distinta) métodos de la superclase. Este es el concepto de redefinición o sobrescritura (override) de métodos. El método debe tener los mismos tipos de argumentos y tipo de retorno.

El modificador **`override`** es necesario para ampliar o modificar la implementación abstracta o virtual de un método.

No se puede sobrescribir un método estático o no virtual. El método base sobrescrito debe ser `virtual`, `abstract` u `override`.

Ejemplo:

```
class TestOverride {
    public class Employee {
        public string name;

        // Basepay is defined as protected, so that it may be
        // accessed only by this class and derived classes.
        protected decimal basepay;

        // Constructor to set the name and basepay values.
        public Employee(string name, decimal basepay) {
            this.name = name;
            this.basepay = basepay;
        }

        // Declared virtual so it can be overridden.
        public virtual decimal CalculatePay() {return basepay;}
    }
```

```
    // Derive a new class from Employee.
    public class SalesEmployee : Employee {
        // New field that will affect the base pay.
        private decimal salesbonus;

        // The constructor calls the base-class version, and
        // initializes the salesbonus field.
        public SalesEmployee(string name, decimal basepay,
                  decimal salesbonus) : base(name, basepay) {
            this.salesbonus = salesbonus;
        }

        // Override the CalculatePay method
        // to take bonus into account.
        public override decimal CalculatePay() {
            return basepay + salesbonus;
        }
    }

    static void Main() {
        // Create some new employees.
        var employee1 = new SalesEmployee("Alice", 1000, 500);
        var employee2 = new Employee("Bob", 1200);

        Console.WriteLine($"Employee1 {employee1.name} earned:
             {employee1.CalculatePay()}");
        Console.WriteLine($"Employee2 {employee2.name} earned:
             {employee2.CalculatePay()}");
    }
}
/*
    Output:
    Employee1 Alice earned: 1500
    Employee2 Bob earned: 1200
*/
```

La palabra clave **virtual** se usa para modificar una declaración de método (está implementado) y permitir que se sobrescriba en una clase derivada. Por defecto, los métodos son no virtuales, por lo que no se puede sobrescribir un método no virtual. En la clase derivada, se indicar la palabra reservada `override` cuando se redefine el método virtual de la clase base.

new oculta explícitamente un método heredado de una clase base. Cuando se oculta, la versión derivada reemplaza a la versión de la clase base. Es un error usar `new` y `override` en el mismo miembro, porque los dos modificadores tienen significados mutuamente excluyentes. El modificador `new` crea un nuevo método con el mismo nombre y oculta el miembro original. El modificador `override` amplía la implementación de un método heredado.

Ejemplo:

```
public class BaseC {
    public class NestedC {
        public int x = 200;
        public int y;
    }
}
```

```
public class DerivedC : BaseC {
    // Nested type hiding the base type members.
    new public class NestedC {
        public int x = 100;
        public int y;
        public int z;
    }

    static void Main() {
        // Creating an object from the overlapping class:
        NestedC c1 = new NestedC();

        // Creating an object from the hidden class:
        BaseC.NestedC c2 = new BaseC.NestedC();

        Console.WriteLine(c1.x);
        Console.WriteLine(c2.x);
    }
}
/*
Output:
100
200
*/
```

3. Clases abstractas

El modificador **abstract** en una declaración de clase se usa para indicar que una clase está diseñada como clase base de otras clases, no para crear instancias por sí misma.

Ejemplo:

```
abstract class Figura {}
```

4. Métodos abstractos

Método de una clase que no es implementado, se indica mediante la palabra reservada **abstract**.

Si una clase tienen algún método `abstract`, la clase debe ser obligatoriamente **abstract**.

Las clases derivadas (hijas) tienen la obligación de implementar el método (o declararse `abstract`).

Ejemplo:

```
abstract class Shape {
    public abstract int GetArea();
}
```

Ejemplo:

```
class Square : Shape {
    int side;

    public Square(int n) => side = n;

    // GetArea method is required to avoid a compile-time error.
    public override int GetArea() => side * side;

    static void Main() {
        var sq = new Square(12);
        Console.WriteLine($"Area of the square = {sq.GetArea()}");
    }
}
// Output: Area of the square = 144
```

5. Clases y métodos no heredables

La palabra reservada **sealed** (sellada) permite:

- En una clase: impide que otras clases hereden de ella.
- En un método: impide que se puede redefinir dicho método.

La sintaxis es:

```
class A {}
sealed class B : A {}
```

8. INTERFACES

1. Definición de una interface

Los interfaces constituyen un mecanismo para separar interfaz (declaración del método) de implementación (código). Se declaran de manera similar a las clases:

```
interface nombre {…}
```

Un interfaz únicamente puede tener firmas de métodos (sólo la declaración, sin cuerpo), constantes y propiedades.

```
public interface IStringList
{
    void Add(string s);
    int Count { get; }
    string this[int index] { get; set; }
}
```

Ejemplo: definición de una interface solo con propiedades e implementación de la misma por una clase.

```
interface ISampleInterface {
    void SampleMethod();
}
```

```
class ImplementationClass : ISampleInterface {
    // Explicit interface member implementation:
    void ISampleInterface.SampleMethod()
    {
        // Method implementation.
    }

    static void Main()
    {
        // Declare an interface instance.
        ISampleInterface obj = new ImplementationClass();

        // Call the member.
        obj.SampleMethod();
    }
}
```

Ejemplo: definición de una interface solo con propiedades e implementación de la misma por una clase.

```
interface IPoint {
    // Property signatures:
    int X {
        get;
        set;}

    int Y {
        get;
        set;}

    double Distance {
        get;}
}
```

```
class Point : IPoint {
    // Constructor:
    public Point(int x, int y) {
        X = x;
        Y = y;}

    // Property implementation:
    public int X { get; set; }
    public int Y { get; set; }

    // Property implementation
    public double Distance =>
        Math.Sqrt(X * X + Y * Y);
}
```

2. Herencia en interfaces

Entre las interfaces existe una jerarquía que permite **herencia simple y múltiple**.

Ejemplo: definición de una interface que hereda de dos interfaces

```
interface IPrueba : IBase1, IBase2 {...}
```

3. Interfaces genéricas

De la misma manera que se pueden definir clases genéricas, este concepto está soportado también para las interfaces.

```
interface nombre<T> {…}
```

Ejemplo:

```
interface IEquatable<T>
{
    bool Equals(T obj);
}
```

9. EXCEPCIONES

Las excepciones son tipos que derivan de **`System.Exception`**.

El control de excepciones usa las palabras clave `try`, `catch` y `finally` para intentar realizar acciones que pueden no completarse correctamente, para controlar errores cuando se decide que es razonable hacerlo y para limpiar recursos más adelante. Las excepciones se crean mediante el uso de la palabra clave `throw`.

Cuando se genera una excepción, CLR desenreda la pila, busca un método con un bloque `catch` para el tipo de excepción específico y ejecuta el primer bloque `catch` que encuentra. Si no encuentra ningún bloque `catch` adecuado en cualquier parte de la pila de llamadas, finalizará el proceso y mostrará un mensaje al usuario.

El código de un bloque `finally` se ejecuta incluso si se produce una excepción.

Ejemplo:

```
public class ExceptionTest {
    static double SafeDivision(double x, double y) {
        if (y == 0)
            throw new DivideByZeroException();
        return x / y;
    }

    public static void Main() {
        // Input for test purposes. Change the values to see
        // exception handling behavior.
        double a = 98, b = 0;
        double result;

        try {
            result = SafeDivision(a, b);
            Console.WriteLine("{0} divided by {1} = {2}", a, b, result);
        }
        catch (DivideByZeroException) {
            Console.WriteLine("Attempted divide by zero.");
        }
    }
}
```

1. Creación de una excepción

Son clases que derivan de `Exception`.

```
class CustomException : Exception
{
    public CustomException(string message) { }
}
```

Ejemplo:

```
private static void TestThrow() {
    throw new CustomException("Custom exception in TestThrow()");
}
```

Tipos de excepciones destacadas:

- **DivideByzeroException**: división por cero.
- **IndexOutOfRangeException**: índice fuera de rango.
- **NullReferenceException**: referencia a null (en vez de a un objeto).
- **StackOverflowException**: desbordamiento de pila.



10. ESPACIOS DE NOMBRES

La palabra clave **namespace** se usa para declarar un ámbito que contiene un conjunto de objetos relacionados (misma funcionalidad que package en Java). Se puede usar un espacio de nombres para organizar los elementos de código.

```
namespace SampleNamespace {
    class SampleClass { }

    interface ISampleInterface { }

    struct SampleStruct { }

    enum SampleEnum { a, b }

    namespace Nested {
        class SampleClass2 { }
    }
}

namespace N1.N2 {
    class A {}
    class B {}
}
```

La directiva **using** permite usar tipos definidos en un espacio de nombres. Por defecto, esta directiva importa todos los tipos del espacio de nombres.

```
using System;
```

Las directivas using **implícitas** implica que el compilador añade automáticamente un conjunto de directivas `using` en función del tipo de proyecto. Por ejemplo, en el caso de las aplicaciones de consola, las siguientes directivas se incluyen de forma implícita en la aplicación:

```
using System;
using System.IO;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
```

Colecciones (`System.Collections.Generic`):

- `HashSet<T>`
- `Dictionary<TKey,TValue>`
- `SortedSet<T>`
- `List<T>`
- `Stack<T>`
- `LinkedList<T>`
- `Queue<T>`

11. ENTRADA / SALIDA

La clase **Console** (ubicada en el namespace System) representa los flujos de entrada, salida y error estándar para las aplicaciones de consola. Esta clase no puede heredarse.

Métodos:

- **Console.Read()**: lee el siguiente carácter del flujo de entrada estándar.
- **Console.ReadLine()**: lee la siguiente línea de caracteres del flujo de entrada estándar.
- **Console.Write(String)**: escribe la cadena en el flujo de salida estándar.
- **Console.WriteLine(String)**: escribe la cadena seguido de fin de línea en el flujo de salida estándar.

Ejemplos:

```
using System;

public class Example {
    public static void Main()
    {
        Console.Write("Hello ");
        Console.WriteLine("World!");
        Console.Write("Enter your name: ");
        String name = Console.ReadLine();
        Console.Write("Good day, ");
        Console.Write(name);
        Console.WriteLine("!");
    }
}
// The example displays output similar to the following:
//       Hello World!
//       Enter your name: James
//       Good day, James!



Decimal pricePerOunce = 17.36m;
String s = String.Format("The current price is {0} per ounce.",
                         pricePerOunce);
Console.WriteLine(s);
// Result: The current price is 17.36 per ounce.


string s = String.Format("At {0}, the temperature is {1}°C.",
                         DateTime.Now, 20.4);
Console.WriteLine(s);
// Output similar to: 'At 4/10/2015 9:29:41 AM, the temperature is 20.4°C.'
```